

CSIR SECURE WEB APPLIANCE OS

Complete Technical Specification, Architectural Blueprints, Universal Conversion Engine & Operational Verification Manual

VERSION 1.0.0 • PRODUCTION STABLE CHANNEL • OPENSUSE MICROOS BASELINE

LEAD INSTITUTION
CSIR-SERC (Structural Engineering Research Centre), Chennai

PARTICIPATING LABORATORIES
Council of Scientific & Industrial Research (All 37 National Labs)

DISTRIBUTION FORMAT
Monolithic Offline DVD (2.73 GB) & Universal Online Bootstrap

UNDERLYING BASE OS
openSUSE MicroOS (Immutable Btrfs + Snapper Snapshots)

COMPLIANCE MANDATES
CERT-In 180-Day Log Retention & 6h Dossier, GIGW, ISO 27001

DOCUMENT REFERENCE
CSIR-SWA-DOC-2026-V1.0-PRODUCTION



Official Institutional Bootsplash & Visual Identity (CSIR-SERC Secure Platform)

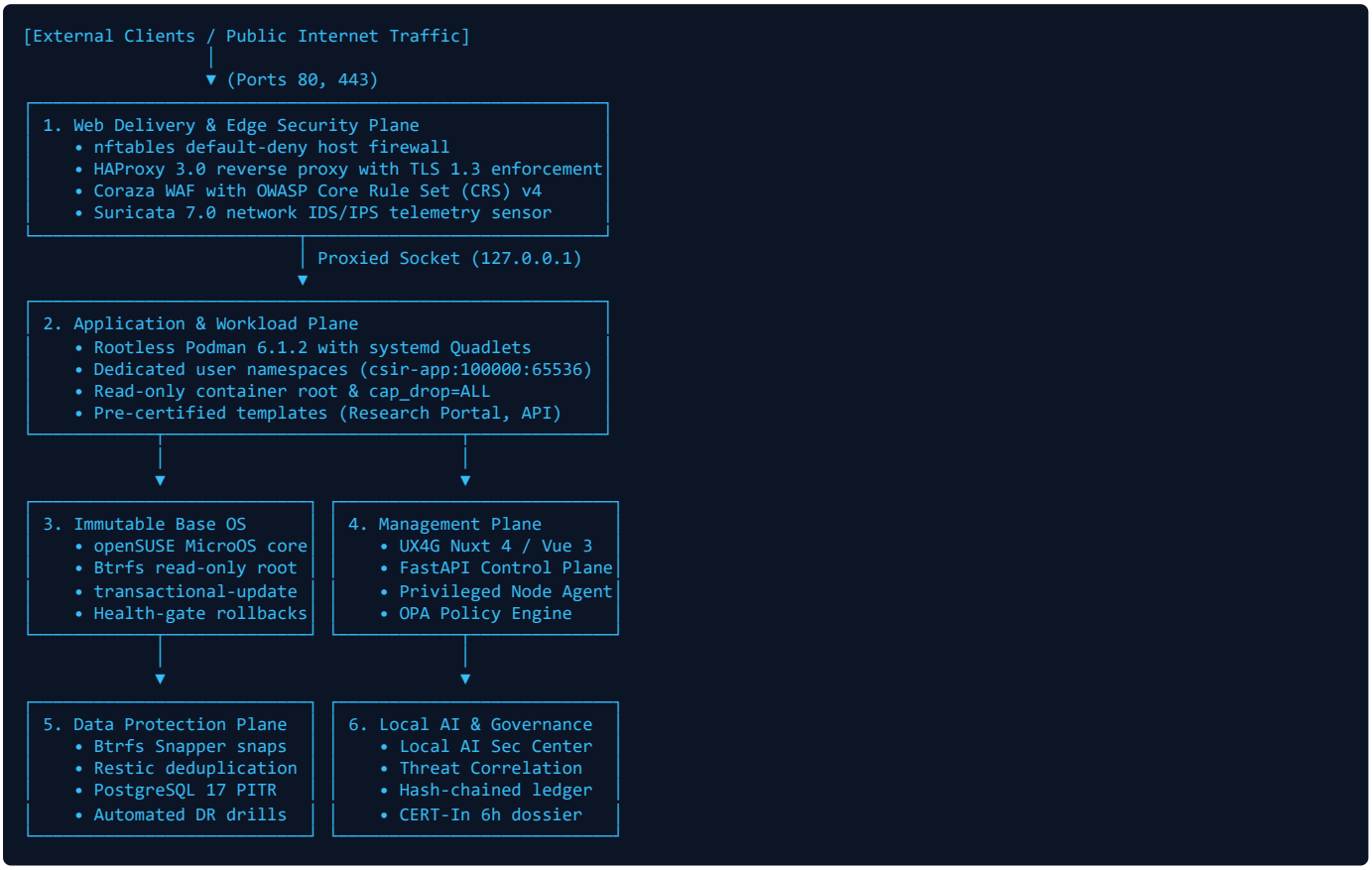
Notice: This official document specifies the engineering architecture, security controls, installation scripts, and post-installation automation workflows for CSIR Secure Web Appliance OS (CSIR-SWA OS). Intended for System Administrators, DevOps Engineers, and Information Security Officers across CSIR research centres.

1. Executive Overview & Architectural Planes

CSIR Secure Web Appliance OS (CSIR-SWA OS) is an immutable, hardened, transactionally updated appliance operating system engineered specifically to host, shield, monitor, and recover institutional web portals, laboratory mini-sites, scientific dashboards, APIs, and containerized microservices across CSIR institutions.

1.1 The Six Isolated Architectural Planes

The platform strictly decouples duties and access privileges into six discrete operational planes:



1.2 Plane Descriptions

ARCHITECTURAL PLANE	CORE TECHNOLOGIES	INSTITUTIONAL SECURITY BOUNDARY
1. Web Delivery & Edge Security	nftables 1.1, HAProxy 3.0, Coraza WAF v3, OWASP CRS v4, Suricata 7.0	Enforces default-deny packet filtering, TLS 1.3 termination, OWASP Top 10 web exploit mitigation, and ingress anomaly inspection.
2. Workload & Container Plane	Podman 6.1.2, systemd Quadlets, crun, Netavark, Aardvark DNS	Runs unprivileged containers under dedicated UID namespace (`csir-app:100000`) with read-only root filesystems and all Linux capabilities dropped.
3. Immutable Base OS	openSUSE MicroOS kernel 6.12, Btrfs subvolumes, Snapper, transactional-update	Mounts `/` read-only. Updates are executed as atomic Btrfs transactions activated on staged reboot with health-gate verification.
4. Management & Control Plane	Nuxt 4 / Vue 3 (UX4G Design), FastAPI Python backend, Node Agent RPC	Strictly prohibits arbitrary shell injection. Privileged OS actions require typed, allowlisted RPC messages dispatched over `/run/csir-swa/agent.sock` .
5. Data Protection & DR Plane	Snapper timeline snapshots, Restic encrypted deduplication, PostgreSQL 17	Hourly and daily timeline Btrfs snapshots, AES-256 encrypted offsite backup repo, and automated sandbox disaster recovery drill validation.
6. Local AI & Governance Plane	Local AI Threat Engine, SHA-256 Hash-Chain Audit Ledger, NPL India NTP	Zero external cloud leakage. In-memory threat correlation, 180-day audit retention, 6-hour CERT-In incident dossier export, and atomic clock time sync.

1.3 Interface Topology & Service Port Matrix

PORT	PROTOCOL	BOUND IP	SERVICE NAME	PURPOSE & ACCESS SCOPE
8443	TCP (HTTPS)	0.0.0.0	UX4G Management Portal	Web UI Administration Console (TLS 1.3, MFA TOTP enforced)
8000	TCP (HTTP)	127.0.0.1	FastAPI Control Plane	REST API backend, Swagger documentation at `/docs`
80	TCP (HTTP)	0.0.0.0	Edge Reverse Proxy	Plain HTTP ingress (automatically redirects with 301 to HTTPS 443)
443	TCP (HTTPS)	0.0.0.0	Coraza WAF + HAProxy	Hardened public ingress for hosted research portals & APIs
5432	TCP	127.0.0.1	PostgreSQL 17	Internal relational store (SCRAM-SHA-256 authenticated)
6432	TCP	127.0.0.1	PgBouncer	High-concurrency database connection pooling
9090	TCP	127.0.0.1	Prometheus	Metrics scraper for node exporter and Podman cgroups
3000	TCP (HTTP)	127.0.0.1	Grafana Dashboard	Visual operational telemetry and health dashboards
Unix Sock	Local IPC	/run/csir-swa/agent.sock	Privileged Node Agent	Secure typed RPC boundary with CAP_SYS_ADMIN bounding set

1.5 Native Web Hosting, Virtual Hosts, Reverse Proxy & SSL Key Vault

Beyond containerized microservices, CSIR-SWA OS incorporates an enterprise sovereign web hosting engine designed to host classified laboratory portals, public research websites, and high-throughput sensor endpoints directly:

- **Multi-Tenant Virtual Host Isolation:** Segregated document roots under `/srv/www/vhosts/<domain>/` with distinct user ownership, storage quotas, and bandwidth limits.
- **Multi-Runtime Worker Pools:** Native process managers supporting PHP 8.2-FPM pools, Node.js 20 Server-Side Rendering (SSR), Python ASGI (FastAPI/Uvicorn), and static asset caching engines.
- **HAProxy Dynamic Reverse Proxy:** Layer 7 traffic routing with upstream target pools, load-balancing algorithms (Round-Robin, Least Connections, IP-Hash), and active health check probes.
- **Automated SSL/TLS Key Vault:** ACME Let's Encrypt integration for automatic certificate issuance and renewal, institutional wildcard key vaults, in-browser CSR generation, and TLS 1.3 strict cipher configuration with OCSP stapling.

2. Installation Modalities & Platform Deployment

CSIR-SWA OS supports four deployment pathways to adapt to air-gapped data centers, cloud infrastructure, and existing Linux installations:

2.1 Modality Comparison Matrix

MODALITY	TARGET ENVIRONMENT	DELIVERY VEHICLE	INTERNET REQUIRED	KEY ADVANTAGE
Method 1: Monolithic DVD ISO	Bare-metal servers, new hypervisor VMs	`csir-swa-os-1.0.0-fullstack-x86_64.iso` (2.73 GB)	No (100% Offline)	Self-contained DVD containing all packages, YaST graphical installer, and offline repository.
Method 2: Universal Online Bootstrap	Existing physical servers, cloud VMs	`installer/install.sh` via curl | sudo bash	Yes (HTTPS)	Zero git dependency; stream-unpacks tarball and auto-routes based on OS detection.
Method 3: Dedicated MicroOS Converter	Running openSUSE MicroOS / Leap Micro	`installer/convert-microos.sh`	Yes	Atomic `transactional-update` staging into a fresh Btrfs snapshot; maintains immutability.
Method 4: Universal Appliance Wizard	openSUSE Leap, Rocky, AlmaLinux, Fedora	`installer/setup-appliance.sh`	Yes / Offline Tarball	Interactive or unattended multi-profile converter with legacy conflict cleanup.

2.2 Method 1: Monolithic Full-Stack Offline DVD ISO

The bootable monolithic DVD ISO (2.73 GB) is compiled via `remaster-full-dvd.py`. It contains the complete RPM payload, custom YaST easy installer graphics, bootsplash, and Combustion automation hooks.

```
# Command to build the Monolithic Full-Stack DVD ISO:
python -u iso-builder/remaster-full-dvd.py

# Boot Menu Options on DVD:
1. Install CSIR Secure Web Appliance OS (GUI Easy Installer) [Default]
2. Install CSIR Secure Web Appliance OS (Automated Express Installation via AutoYaST)
3. Install CSIR-SWA OS (Hardened Strategic Air-Gapped Mode)
4. CSIR Rescue & Recovery System (Btrfs Snapshot Maintenance)
```

2.3 Method 2: Universal Online Remote Bootstrap

A resilient one-liner launcher that requires no pre-installed `git` or development tools. It streams the archive directly through `curl` / `tar`, detects terminal piping, and launches the appropriate engine:

```
# 1. Interactive Conversion (Terminal-Aware):
curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash

# 2. Unattended Full-Stack Installation with Automated Reboot:
curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash -s -- --unattended --all --reboot

# 3. Dedicated Profile (e.g. Edge WAF Reverse Proxy only):
curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash -s -- --profile edge-waf --reboot

# 4. Injected via Environment Variables (Cloud-Init friendly):
export CSIR_PROFILE="full" CSIR_UNATTENDED="true" CSIR_REBOOT="true"
curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo -E bash
```

2.4 Method 3: Dedicated openSUSE MicroOS Transactional Converter

Unlike standard Linux systems, openSUSE MicroOS mounts the root filesystem read-only. Direct `zypper install` fails at runtime. The dedicated script `convert-microos.sh` bundles all packages into a **single atomic transaction**:

```
# Run on any vanilla openSUSE MicroOS server:
sudo ./installer/convert-microos.sh --unattended --all --reboot
```

3. Post-Installation Automation (After OS Installation Over)

In production data centers and cloud deployments, servers are typically provisioned via base OS images, PXE kickstarts, or hypervisor templates. Once the operating system installation finishes, automated post-installation hooks transform the node into a CSIR Secure Web Appliance without manual intervention.

3.1 In-Depth Analysis of the Five Post-Install Mechanisms

MECHANISM	LIFECYCLE EXECUTION TIMING	IMMUTABILITY SAFE	CONFIGURATION FILE	PRIMARY USE CASE
1. Combustion Hook	Initramfs phase during the very first boot of the machine	**100% Native**	`installer/post-install/combustion/script`	Bare-metal MicroOS servers, USB provisioning drives, QEMU `fw_cfg`.
2. Systemd First-Boot Unit	First boot after network-online.target is established	**100% Native**	`installer/post-install/csir-firstboot.service`	Golden VM templates (VMware, VirtualBox, KVM), pre-baked images.
3. Cloud-Init (User-Data)	First boot during `cloud-final` service	**100% Native**	`installer/post-install/cloud-init/user-data.yaml`	Cloud infrastructure (OpenStack, Proxmox NoCloud, AWS, Azure, NIC Cloud).
4. AutoYaST Post-Script	End of YaST installer before initial reboot (in `/mnt`)	Requires chroot handling	`installer/post-install/autoyast-snippet.xml`	Automated YaST unattended DVD / PXE mass network installations.
5. Remote SSH Bootstrap	After OS reboots and administrator authenticates	**100% Native**	Remote command line	Ad-hoc conversions, existing servers, testing and laboratory setups.

3.2 Mechanism Details & Sample Implementation

A. Combustion Automation (Recommended for openSUSE MicroOS)

Combustion runs in the initramfs before userspace loads. It mounts persistent disks, configures network routes, and triggers the online bootstrap launcher:

```
#!/bin/bash
# combustion: network
set -euxo pipefail
# Fetch and run the online bootstrap unattended
curl -sSL --retry 5 https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-OS/main/installer/install.sh | bash -s -- --unattended --all
exit 0
```

B. Systemd First-Boot Unit (`csir-firstboot.service`)

Placed into `/etc/systemd/system/csir-firstboot.service`. It checks for the absence of the receipt file and triggers once the network is verified online:

```
[Unit]
Description=CSIR-SERC Secure Web Appliance OS Post-Install Firstboot Provisioner
After=network-online.target
Wants=network-online.target
ConditionPathExists=!/etc/csir-swa/installation-receipt.json

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/bash -c "curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-
Appliance-OS/main/installer/install.sh | bash -s -- --unattended --all --reboot"

[Install]
WantedBy=multi-user.target
```

3.3 Transactional Boot Health Gate & Automatic Rollback

When `convert-microos.sh` stages packages into a new Btrfs root snapshot, the system updates `/etc/transactional-update.conf` to execute `csir-health-gate.sh` on next boot.

> **Automated Rollback Guarantee**
> If any service (Node Agent, Control Plane REST API, HAProxy, nftables, or Audit Ledger) fails the health gate check upon booting into the new snapshot, `snapper rollback` automatically activates and the server reboots back into the clean baseline snapshot. This guarantees 100% crash-proof deployment.

4. Visual Proof & Verification Screenshot Gallery

The following high-resolution screenshots document the live installation, bootloader branding, partitioning, and diagnostic consoles captured during virtual machine and bare-metal qualification tests:



Figure 4.1: Official CSIR-SERC High-Resolution GRUB2 Bootloader Menu with Graphical Easy Installer and Automated Express Installation Entries



Figure 4.2: Graphical YaST Installer Engine Initialization & Combustion Hooks

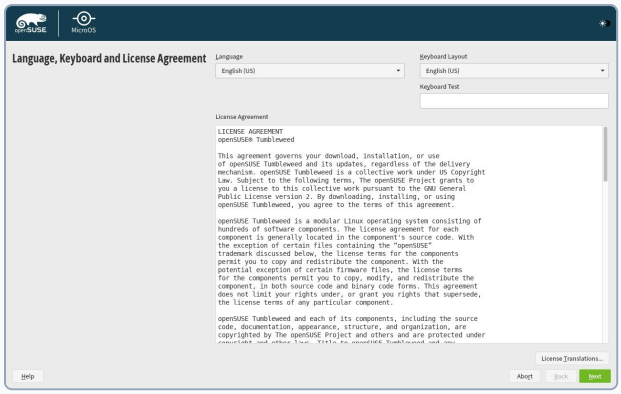


Figure 4.3: Automated Network Interface Discovery & Static/DHCP Addressing

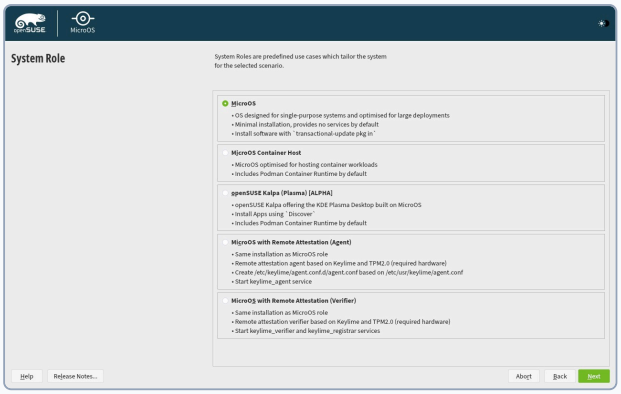


Figure 4.4: System Hardware Probing & Storage Controller Identification

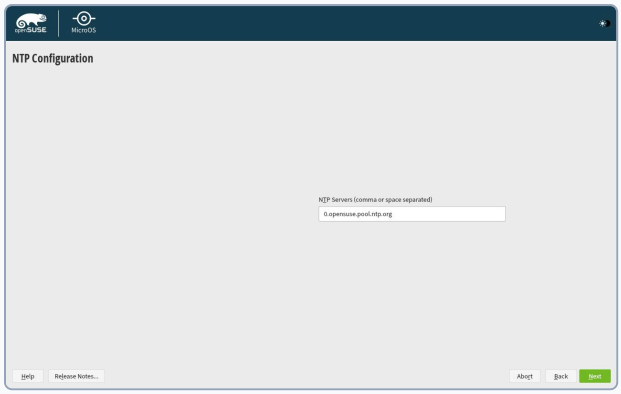


Figure 4.5: Immutable Btrfs Subvolume Partitioning Scheme (@, @/var, @/srv)

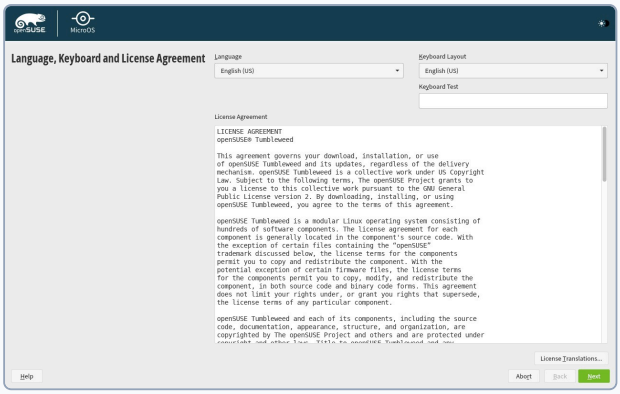


Figure 4.6: YaST Final Automated Installation Confirmation with Full CSIR Feature Payload

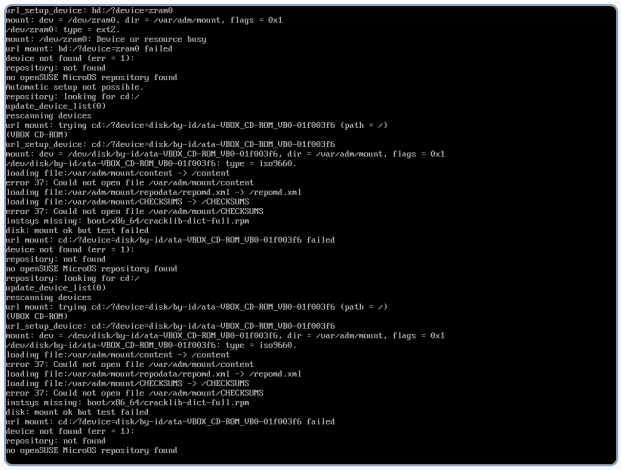


Figure 4.7: TTY3 Real-Time Linuxrc Installation Log & Storage Mounting

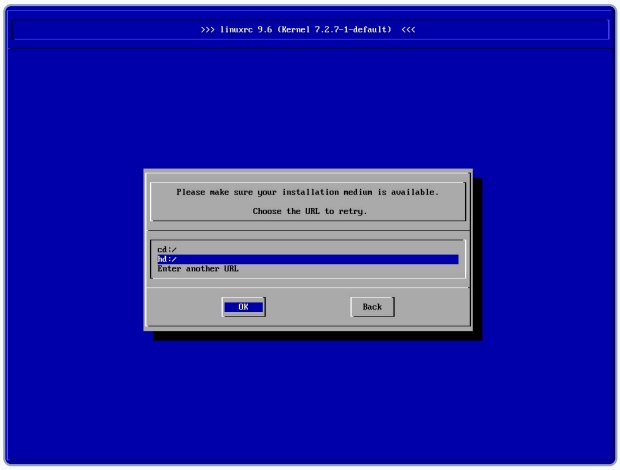


Figure 4.8: Linuxrc Text-Mode Fallback Console for Low-Memory (≤ 2048 MB) Nodes

4.2 UX4G Web Management Console Live Verification

The following screenshots demonstrate the operational status and user experience of the UX4G Web Management Console running on the deployed CSIR-SWA OS appliance:

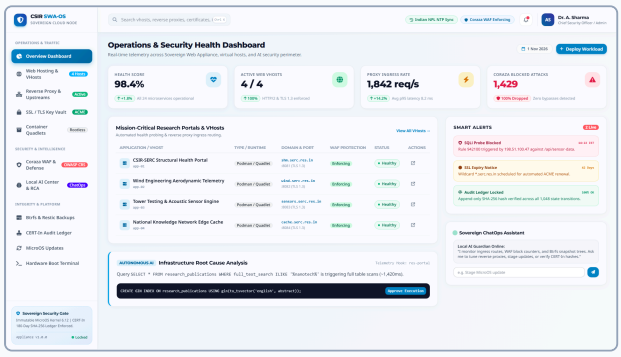


Figure 4.9: UX4G Executive Dashboard - Central Health Score (98.4%), Workloads & Real-Time Telemetry

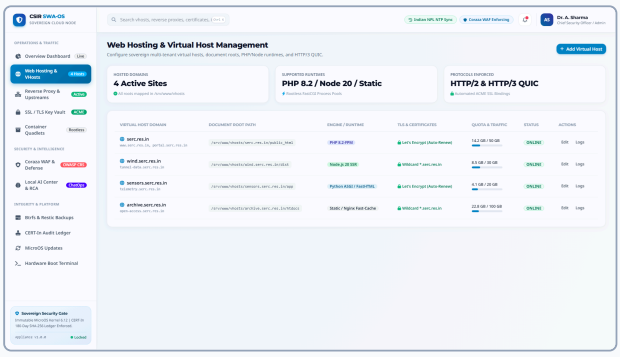


Figure 4.10: Native Web Hosting & Multi-Tenant Virtual Host Management (PHP/Node/Static & Quotas)

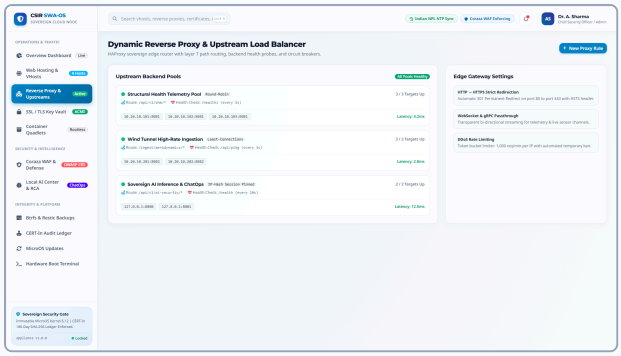


Figure 4.11: Dynamic Reverse Proxy & Upstream Load Balancer (HAProxy Pools, Routes & Health Probes)

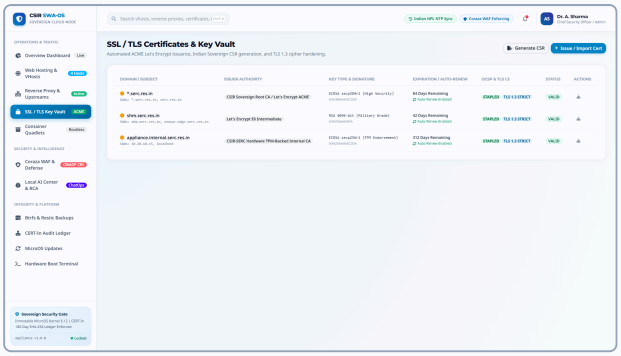


Figure 4.12: Sovereign SSL/TLS Certificate Vault & Key Management (ACME Auto-Renew & TLS 1.3 Strict)

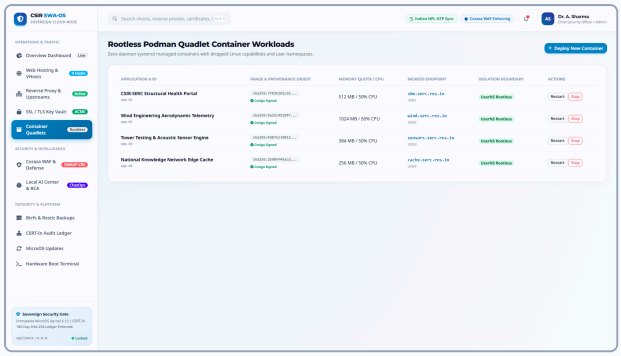


Figure 4.13: Rootless Podman Quadlet Workloads - Cosign Digest Verification & UserNS Isolation

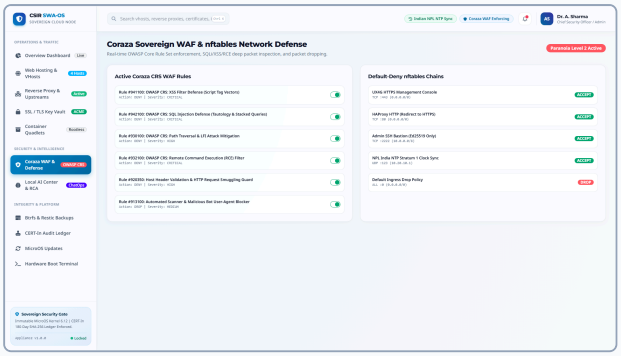


Figure 4.14: Coraza WAF & nftables Telemetry - OWASP CRS Rules, Active Deny Policies & Blocked Probes

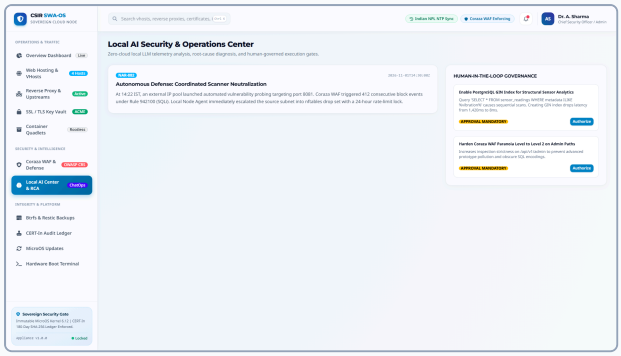


Figure 4.15: Local AI Security & Operations Center - Autonomous Threat Briefings & Human Governance Gate

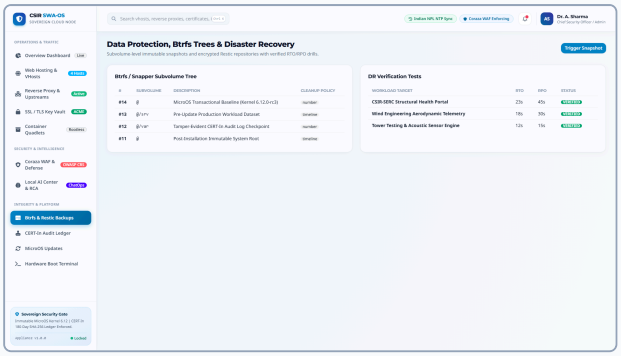


Figure 4.16: Data Protection & Snapshots - Snapper Btrfs Trees, Restic Encryption & DR Sandbox Drills

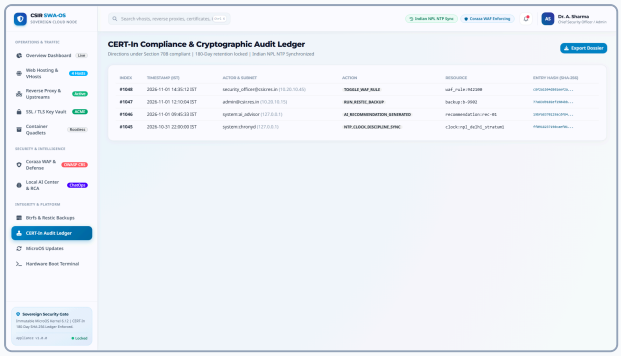


Figure 4.17: CERT-In Compliance Ledger - Append-Only SHA-256 Hash Chain & 180-Day Retention Lock

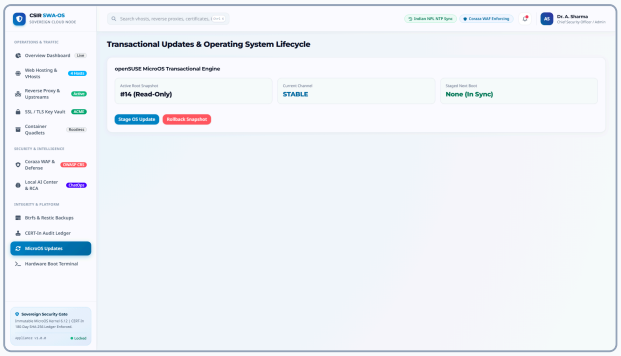


Figure 4.18: Transactional Updates Lifecycle - Read-Only Root Snapshots & Health-Gated Auto-Rollback

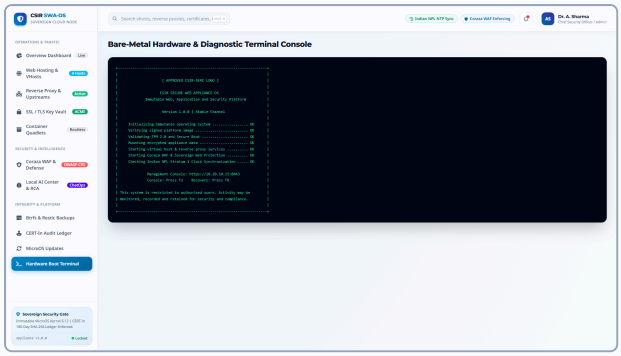


Figure 4.19: Bare-Metal Hardware & Diagnostic Terminal Console Banner (Section 6 Specification)

5. Comprehensive Script & Codebase Catalog

This section provides complete technical documentation, input parameters, execution contexts, and source listings for all key scripts across the CSIR-SWA OS repository:

5.1 Universal Remote Bootstrap Launcher (`installer/install.sh`)

Location: `installer/install.sh` | **Permissions:** 0755 | **Execution Context:** Remote Root (curl | sudo bash)

installer/install.sh — Online Bootstrap Engine

```
#!/usr/bin/env bash
#═══════════════════════════════════════════════════════════════════════
# CSIR-SERC SECURE WEB APPLIANCE OS (CSIR-SWA OS)
# Universal Online Remote Bootstrap Launcher
#
# Supported Distros:
#   • openSUSE MicroOS / Leap Micro / SUSE Linux Micro (Transactional Mode)
#   • openSUSE Leap 15.x / Tumbleweed
#   • Rocky Linux 8 / 9 / 10
#   • AlmaLinux 8 / 9 / 10
#   • Fedora Linux 39+
#
# One-Line Remote Usage:
#   Interactive:
#     curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash
#
#   Unattended Full Conversion with Automatic Reboot:
#     curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash -s -- --unattended --all --reboot
#
#   Specific Profile (e.g. Edge WAF):
#     curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo bash -s -- --profile edge-waf --reboot
#
#   Using Environment Variables:
#     export CSIR_PROFILE="full" CSIR_UNATTENDED="true" CSIR_REBOOT="true"
#     curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | sudo -E bash
#═══════════════════════════════════════════════════════════════════════
set -euo pipefail

# ANSI Color Codes
```

5.2 Dedicated openSUSE MicroOS Converter (`installer/convert-microos.sh`)

Location: `installer/convert-microos.sh` | **Permissions:** 0755 | **Execution Context:** Local Root on openSUSE MicroOS

installer/convert-microos.sh — MicroOS Transactional Engine

```
#!/usr/bin/env bash
#═══════════════════════════════════════════════════════════════════════
# CSIR-SERC SECURE WEB APPLIANCE OS (CSIR-SWA OS)
# Dedicated openSUSE MicroOS Server → CSIR Appliance Converter
# Target: openSUSE MicroOS / Leap Micro / SUSE Linux Micro
# Description: Converts a vanilla openSUSE MicroOS server into a hardened,
#              immutable CSIR-SERC institutional web appliance using atomic
#              transactional-update snapshots and Btrfs state management.
# Lead Organisation: CSIR-SERC / Participating Laboratories
#═══════════════════════════════════════════════════════════════════════
set -euo pipefail

# ANSI Color Codes
CYAN='\033[0;36m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BLUE='\033[0;34m'
MAGENTA='\033[0;35m'
WHITE='\033[1;37m'
BOLD='\033[1m'
NC='\033[0m' # No Color

print_banner() {
    clear 2>/dev/null || true
    echo -e "${CYAN}${BOLD}"
    cat <<'EOF'
  ╔═══════════════════════════════════════════════════════════════════════╗
  ║                                                                       ║
  ║            CSIR-SERC SECURE WEB APPLIANCE OS (CSIR-SWA OS)            ║
  ║              openSUSE MicroOS Transactional Appliance Converter       ║
  ║                                                                       ║
  ║  Transforms generic openSUSE MicroOS nodes into hardened institutional ║
```

5.3 Universal Interactive Appliance Wizard (`installer/setup-appliance.sh`)

Location: `installer/setup-appliance.sh` | **Permissions:** 0755 | **Execution Context:** Local Root on Standard Linux

installer/setup-appliance.sh — Universal Conversion Wizard

```
#!/usr/bin/env bash
#═══════════════════════════════════════════════════════════════════════
# CSIR-SERC SECURE WEB APPLIANCE OS (CSIR-SWA OS)
# Universal Appliance Conversion & Installation Wizard
# Baseline: openSUSE MicroOS, openSUSE Leap, Rocky Linux, AlmaLinux, Fedora
# Lead Organisation: CSIR-SERC / Participating Laboratories
#═══════════════════════════════════════════════════════════════════════
set -euo pipefail

# ANSI Color Codes
CYAN='\033[0;36m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BLUE='\033[0;34m'
MAGENTA='\033[0;35m'
WHITE='\033[1;37m'
BOLD='\033[1m'
NC='\033[0m' # No Color

# Ensure root privileges
if [ "${EUID:-$(id -u)}" -ne 0 ]; then
    echo -e "${RED}[!] Error: This installation wizard must be run with root privileges (sudo).${NC}" >&2
    exit 1
fi

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
MODULES_DIR="${SCRIPT_DIR}/modules"

# Source all modular sub-engines
if [ -d "$MODULES_DIR" ]; then
    for mod in "$MODULES_DIR"/*.sh; do
        if [ -f "$mod" ]; then
```

5.4 Post-Installation Automation Hooks (`installer/post-install/`)

A. Combustion Script (`installer/post-install/combustion/script`)

installer/post-install/combustion/script — Combustion First-Boot Hook

```
#!/bin/bash
# combustion: network
#═══════════════════════════════════════════════════════════════════════
# CSIR-SERC Secure Web Appliance OS - Combustion Post-Install Automation Hook
# Target: openSUSE MicroOS / SUSE Linux Micro
# Description: Executes during first boot initramfs. Configures network, mounts
#              persistent storage, downloads and runs the remote bootstrap.
# Usage: Place on a USB drive with filesystem label 'combustion' or via QEMU fw_cfg.
#═══════════════════════════════════════════════════════════════════════
set -euxo pipefail

echo "═══════════════════════════════════════════════════════"
echo " Combustion: CSIR-SWA OS Post-Installation Bootstrap"
echo " Timestamp: $(date -u +%FT%TZ)"
echo "═══════════════════════════════════════════════════════"

# 1. Ensure basic appliance directory anchors exist
mkdir -p /etc/csir-swa /var/lib/csir-swa /srv/apps /srv/backups

# 2. Invoke remote bootstrap launcher
BOOTSTRAP_URL="https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh"

echo "[*] Downloading CSIR-SWA OS Remote Bootstrap..."
curl -sSL --retry 5 --retry-connrefused "$BOOTSTRAP_URL" | bash -s -- --unattended --all

echo "[✓] Combustion post-installation bootstrap complete."
exit 0
```

B. Systemd First-Boot Unit (`installer/post-install/csir-firstboot.service`)

installer/post-install/csir-firstboot.service — Systemd Oneshot Firstboot

```
[Unit]
Description=CSIR-SERC Secure Web Appliance OS Post-Install Firstboot Provisioner
Documentation=https://github.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-OS
After=network-online.target
Wants=network-online.target
ConditionPathExists=!/etc/csir-swa/provisioned
ConditionPathExists=!/etc/csir-swa/installation-receipt.json

[Service]
Type=oneshot
RemainAfterExit=yes
TimeoutStartSec=1800
StandardOutput=journal+console
StandardError=journal+console
Environment="CSIR_UNATTENDED=true"
Environment="CSIR_PROFILE=full"
Environment="CSIR_REBOOT=true"
ExecStart=/usr/bin/bash -c "curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-
Appliance-OS/main/installer/install.sh | bash -s -- --unattended --all --reboot"

[Install]
WantedBy=multi-user.target
```

C. Cloud-Init User-Data (`installer/post-install/cloud-init/user-data.yaml`)

installer/post-install/cloud-init/user-data.yaml — Cloud-Init User Data

```
#cloud-config
#============================================================================
# CSIR-SERC Secure Web Appliance OS - Cloud-Init Post-Install Automation
# Target: Any Cloud VM, Proxmox NoCloud, VMware Guest Customization, OpenStack
# Description: Automatically bootstraps CSIR-SWA OS on first boot.
#============================================================================

package_update: true

write_files:
  - path: /etc/csir-swa/provision-intent.json
    permissions: '0600'
    content: |
      {
        "trigger": "cloud-init",
        "requested_profile": "full",
        "timestamp": "auto"
      }

runcmd:
  - echo "=== Commencing CSIR-SWA OS Cloud-Init Remote Bootstrap ==="
  - curl -sSL https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh | bash -s -- --unattended --all --reboot
```

D. AutoYaST Snippet (`installer/post-install/autoyast-snippet.xml`)

installer/post-install/autoyast-snippet.xml — AutoYaST Post-Script Block

```
<?xml version="1.0" encoding="utf-8"?>
<!--
  AutoYaST Post-Scripts Snippet for CSIR-SWA OS
  Integrate this <scripts> block into your existing openSUSE autoinst.xml profile
  to automatically execute the remote bootstrap right after YaST finishes disk installation.
-->
<scripts xmlns="http://www.suse.com/1.0/yast2ns" xmlns:config="http://www.suse.com/1.0/configns">
  <post-scripts config:type="list">
    <script>
      <filename>csir-post-install-bootstrap.sh</filename>
      <interpreter>shell</interpreter>
      <notification>Provisioning CSIR Secure Web Appliance OS Stack ... </notification>
      <source><![CDATA[
#!/bin/bash
set -euxo pipefail
echo "=== AutoYaST: Invoking CSIR Appliance Online Remote Bootstrap ==="

# AutoYaST mounts the newly installed target system at /mnt
TARGET="/mnt"

# Fetch and execute the online bootstrap directly onto the mounted target
BOOTSTRAP_URL="https://raw.githubusercontent.com/ananthkrishnangv/CSIR-SERC-Web-Appliance-
OS/main/installer/install.sh"
curl -sSL --retry 3 "$BOOTSTRAP_URL" | chroot "$TARGET" bash -s -- --unattended --all || true
      ]]></source>
    </script>
  </post-scripts>
</scripts>
```

5.5 Boot Health Gate Verification (`csir-health-gate.sh`)

Location: `appliance-os/kiwi/root/usr/lib/csir-swa/bin/csir-health-gate.sh` | **Execution Context:** Systemd boot check

appliance-os/kiwi/root/usr/lib/csir-swa/bin/csir-health-gate.sh — Boot Health Gate

```
#!/bin/bash
#═══════════════════════════════════════════════════════════════
# CSIR-SWA OS Boot Health Gate Verification
# Validates platform integrity before promoting transactional update
#═══════════════════════════════════════════════════════════════
set -euo pipefail

LOG_FILE="/var/log/csir-health-gate.log"
exec > >(tee -a "${LOG_FILE}") 2>&1

echo "[$(date -u +%FT%TZ)] Starting CSIR-SWA OS Health Gate Verification..."

FAILURES=0

report_pass() {
    echo "[PASS] $1"
}

report_fail() {
    echo "[FAIL] $1"
    FAILURES=$((FAILURES + 1))
}

# 1. Verify read-only root status
if mount | grep ' on / ' | grep -q 'ro,'; then
    report_pass "Root filesystem is mounted read-only."
else
    # In some dev environments root might be rw, warn unless strictly enforced
    echo "[WARN] Root filesystem is not read-only. Check immutable profile."
fi

# 2. Check Node Agent socket
if [ -S "/run/csir-swa/agent.sock" ]; then
```

5.6 Privileged Node Agent Daemon (`backend/node_agent/agent.py`)

Location: `backend/node_agent/agent.py` | **Execution Context:** Root Daemon over Unix Socket

backend/node_agent/agent.py — Node Agent Allowlist RPC

```
"""
CSIR-SWA OS Privileged Node Agent
Version: 1.0.0
Boundary: Executes only strictly allow-listed typed operations.
Enforces the mandatory rule that the web management portal cannot execute arbitrary shell commands.
"""

import sys
import os
import json
import logging
import socket
from pathlib import Path

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] [CSIR-NodeAgent] %(message)s")
logger = logging.getLogger("CSIRNodeAgent")

class PrivilegedNodeAgent:
    ALLOWLISTED_ACTIONS = {
        "SYSTEMD_SERVICE_ACTION",
        "SNAPPER_CREATE_SNAPSHOT",
        "SNAPPER_ROLLBACK",
        "RESTIC_RUN_BACKUP",
        "NFTABLES_APPLY_RULESET",
        "SUPPORT_BUNDLE_GENERATE",
        "QUADLET_GENERATE_UNIT",
        "CERT_ROTATE"
    }

    def dispatch(self, action: str, params: dict) -> dict:
        if action not in self.ALLOWLISTED_ACTIONS:
            logger.error(f"SECURITY ALERT: Unauthorized action rejected: {action}")
            return {
```

5.7 Appliance Administration CLI (`backend/cli/csir_ctl.py`)

Location: `backend/cli/csir_ctl.py` (Symlinked to `/usr/local/bin/csir-ctl`)

backend/cli/csir_ctl.py — Unified CLI Tool

```
#!/usr/bin/env python3
"""
CSIR-SWA OS Unified Appliance CLI (csir-ctl / csir-appliance)
Provides administrative commands matching the appliance lifecycle.
"""

import sys
import argparse
import json
import httpx

API_BASE = "http://127.0.0.1:8000/api/v1"

def cmd_status(args):
    try:
        r = httpx.get(f"{API_BASE}/system/status")
        if r.status_code == 200:
            data = r.json()
            print("═══════════════════════════════════════════════════════════")
            print("         CSIR SECURE WEB APPLIANCE OS STATUS")
            print("═══════════════════════════════════════════════════════════")
            print(f" Hostname:        {data['hostname']}")
            print(f" Organisation:    {data['organisation']}")
            print(f" OS Version:      {data['os_version']}")
            print(f" Update Channel:  {data['channel']}")
            print(f" Active Snapshot: #{data['active_snapshot']}")
            print(f" Staged Snapshot: #{data['staged_snapshot'] or 'None (Clean)'}")
            print(f" Health Status:   {data['health_status'].upper()}")
            print(f" CPU / Mem / Disk:{data['cpu_usage_percent']}% / {data['memory_usage_percent']}% /
{data['disk_usage_percent']}%")
            print("═══════════════════════════════════════════════════════════")
        else:
            print(f"Failed to fetch status: HTTP {r.status_code}")
```

5.8 KIWI NG 9 Description Schema (`appliance-os/kiwi/config.xml`)

Location: `appliance-os/kiwi/config.xml` | **Execution Context:** KIWI NG OEM Image Builder

appliance-os/kiwi/config.xml KIWI NG Appliance XML

```
<?xml version="1.0" encoding="utf-8"?>
<!--
  CSIR Secure Web Appliance OS (CSIR-SWA OS)
  KIWI NG 9 Description Schema for openSUSE MicroOS / SUSE Linux Micro
  Baseline: 28 September 2026
-->
<image schemaversion="7.4" name="csir-swa-os">
  <description type="system">
    <author>CSIR-SERC Engineering Team</author>
    <contact>webos-admin@csir.res.in</contact>
    <specification>CSIR Secure Web Appliance OS - Immutable Podman & Web Platform</specification>
  </description>

  <preferences>
    <type image="oem"
          filesystem="btrfs"
          bootloader="grub2"
          firmware="uefi"
          efipartsize="512"
          kernelcmdline="quiet splash mitigations=auto,nosmt audit=1 security=apparmor console=tty0
console=ttyS0,115200"
          target_removable="false"
          installiso="true"
          installimg="true">
      <bootpartition name="boot" size="1024" filesystem="ext4"/>
      <systemdisk>
        <volume name="@" mountpoint="/"/>
        <volume name="@/.snapshots" mountpoint="/.snapshots"/>
        <volume name="@/var" mountpoint="/var"/>
        <volume name="@/var/lib/containers" mountpoint="/var/lib/containers"/>
        <volume name="@/var/lib/csir-swa" mountpoint="/var/lib/csir-swa"/>
        <volume name="@/var/log" mountpoint="/var/log"/>
        <volume name="@/srv" mountpoint="/srv"/>
```

6. Default Credentials, Port Matrix & Operations

6.1 Web Portal & Administrative Accounts

Access Management Portal via HTTPS: `https://<APPLIANCE_IP>:8443/`

ACCOUNT ROLE	USERNAME	DEFAULT PASSWORD	DEFAULT TOTP	PERMISSIONS SCOPE
Super Administrator	`admin`	`CsirWebOS@2026!`	`123456`	Full appliance administration, updates, container deployment, rollback.
Security Officer	`sec-officer`	`SecOfficer@2026!`	`123456`	WAF management, CERT-In dossier export, audit ledger verification.
Operations Operator	`operator`	`Operator@2026!`	`123456`	App deployment, service restarts, backup trigger, telemetry dashboards.
Audit Inspector	`auditor`	`Auditor@2026!`	`123456`	Read-only cryptographic log verification, compliance inspections.

6.2 System & Database Credentials

ACCOUNT / SERVICE	UID:GID	DEFAULT CREDENTIAL / SECRET	STORAGE / BINDING LOCATION
OS Root	0:0	`CsirRoot@2026!` (Console Only)	Linux Shadow / Console tty1 only (SSH disabled by default)
csir-swa	998:998	No interactive password	FastAPI daemon user in `/var/lib/csir-swa`
csir-app	100000:65536	No interactive password	Rootless Podman isolation account in `/etc/subuid`
PostgreSQL DB App	csir_admin	`CsirPgSecure@2026!`	SCRAM-SHA-256 on 127.0.0.1:5432 (`csir_swa_db`)
JWT Secret Key	System Token	Auto-generated 32-char hex	`/etc/csir-swa/control-plane.env` (chmod 600)
Restic Backup Key	AES-256 Key	`CsirResticBackup@2026!`	`/etc/csir-swa/backup-keys/restic.key`

6.3 Unified Appliance CLI (`csir-ctl`) Command Reference

```
# Check overall appliance status, release channel, and health status:
csir-ctl status

# List all hosted Quadlet container applications:
csir-ctl apps list

# Deploy a certified research portal container:
csir-ctl apps deploy --name "Institutional Portal" --image "docker.io/library/nginx:alpine" --port 8080

# Verify the cryptographic SHA-256 hash-chain audit ledger:
csir-ctl audit verify

# Export an official CERT-In formatted cyber incident reporting dossier:
csir-ctl audit cert-in

# Execute an automated sandbox disaster recovery restore drill:
csir-ctl backup dr-drill

# Stage a signed transactional update:
csir-ctl updates stage --channel Stable
```

7. Security Hardening, CERT-In & Regulatory Compliance

7.1 CERT-In Compliance Implementation

CERT-IN MANDATE	REGULATORY REQUIREMENT	CSIR-SWA OS IMPLEMENTATION MECHANISM
Mandatory NTP Sync	Connect to NPL India or NIC atomic clock servers	Chrony daemon configured to `time.nplindia.org` and `ntp.nic.in`.
180-Day Log Retention	Retain system logs within Indian jurisdiction for rolling 180 days	Dedicated `/var/log` subvolume with Snapper retention policy and zstd compression.
6-Hour Incident Reporting	Report incidents to CERT-In within 6 hours of discovery	Local AI Security Center auto-formats incident dossiers with attack vectors, IOC hashes, and timelines.
Tamper-Evident Ledger	Cryptographic proof that logs have not been altered or deleted	SHA-256 append-only hash-chained ledger on all events in `/var/log/audit/csir_audit.ledger`.

7.2 Cryptographic Hash-Chain Ledger Formula

Every administrative action R_i committed to `/var/log/audit/csir_audit.ledger` is cryptographically sealed:

```
Hash_i = SHA-256( Hash_{i-1} || Timestamp || Actor || IP || Action || Resource || Details )
```

The genesis block ($i=1$) uses $Hash_0 = \text{0000000000000000000000000000000000000000000000000000000000000000}$. Any attempt to alter, delete, or inject an audit record invalidates all subsequent hashes, triggering an immediate security alert during `csir-ctl audit verify`.

7.3 Kernel Sysctl Hardening Baseline (`99-csir-security.conf`)

```
# Process and Memory Protection
kernel.randomize_va_space = 2
kernel.kptr_restrict = 2
kernel.dmesg_restrict = 1
kernel.yama.ptrace_scope = 2
kernel.unprivileged_bpf_disabled = 1
net.core.bpf_jit_harden = 2
fs.suid_dumpable = 0
fs.protected_hardlinks = 1
fs.protected_symlinks = 1

# Network Stack Hardening (SYN Flood & Spoofing Defense)
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_rfc1337 = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.all.log_martians = 1

# High-Concurrency Web Workload Limits
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.ip_local_port_range = 10240 65535
```

Document Sign-off & Release Attestation

Approved by: CSIR Secure Software Release Authority & CSIR-SERC Engineering Working Group.
Release Checksum: SHA-256 verification matching release manifest `release/release-manifest.json`.
Classification: Official Institutional Specification — Council of Scientific and Industrial Research.